

Exemple, LecteursEtEcrivains (v1)

Fabrice.Kordon@lip6.fr



Vous avez dit «LecteursEtEcrivains»?

2

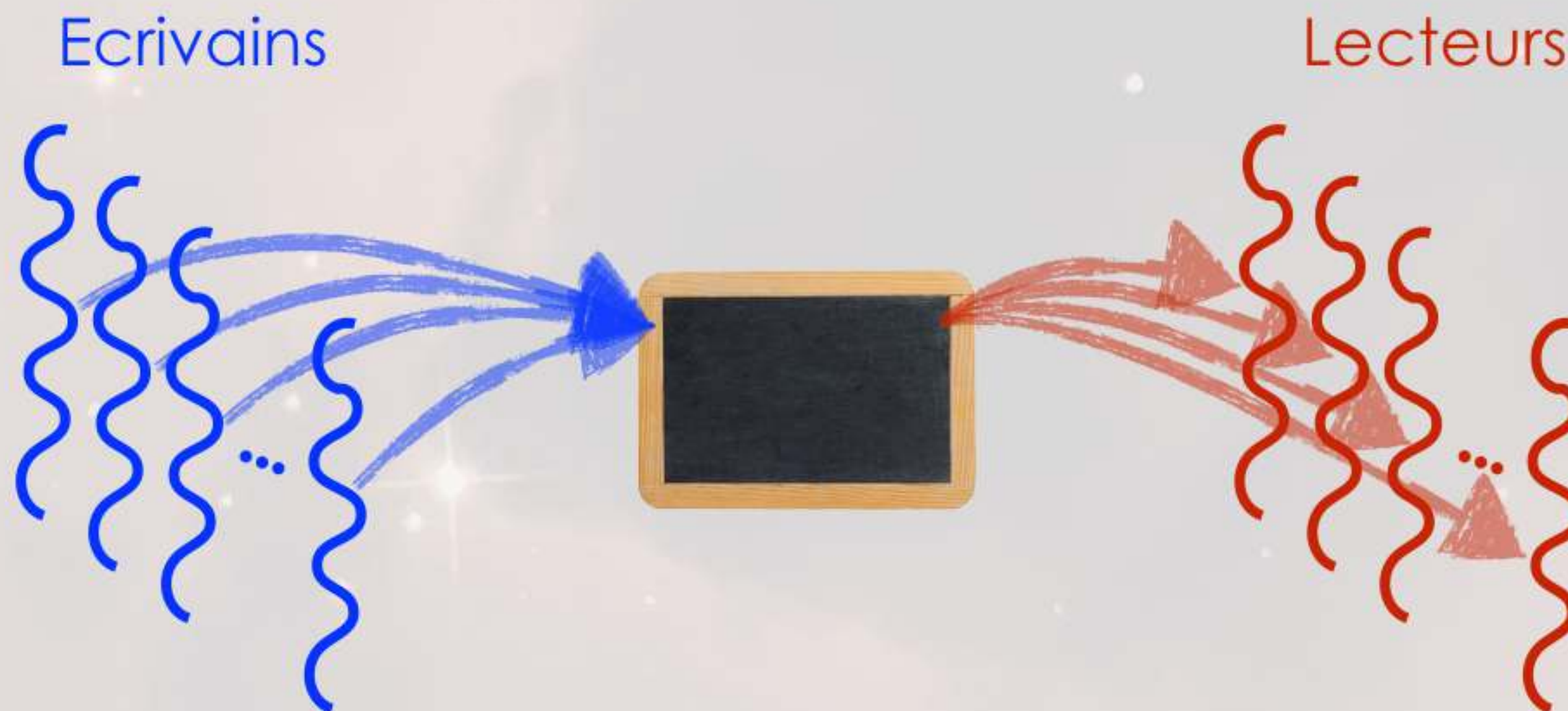
Deux groupes de threads

 Un groupe d'écrivains

▶ 3 écritures d'une valeur aléatoire vers un lecteur choisi aléatoirement

 Un groupe de lecteurs

▶ Recevoir les valeurs transmises



Communication via un «tableau noir»

 Structure de communication fiable (messages non ordonnés)

Implémentation, les «utilitaires»

3

```
import java.util.Random;
import java.util.concurrent.locks.ReentrantReadWriteLock;

@SuppressWarnings("serial") // Eviter le warning sur RienPourMoiException
class RienPourMoiException extends Exception {
    RienPourMoiException (String m) {
        super (m);
    }
}

@SuppressWarnings("serial") // Eviter le warning sur TropPleinException
class TropPleinException extends Exception {
    TropPleinException (String m) {
        super (m);
    }
}
```

La classe UnEcrivain

4

```
class UnEcrivain1 implements Runnable {
    private int monId;
    private int nblecteurs;
    private final TableauNoir1 tn;
    private Random generateur = new Random();
    private static int cpt = 1;
    private final static Object mutex = new Object();

    UnEcrivain1 (TableauNoir1 tab, int nblect) {
        synchronized (mutex) {
            monId = cpt++;
        }
        tn = tab;
        nblecteurs = nblect;
    }

    private void trace (String m) {
        System.out.println ("E " + monId + " : " + m);
        Thread.yield(); // rendre la commutation possible
    }

    private void attendre () {
        try {
            Thread.sleep(500);
        }
        catch (Exception e) {
            System.out.println ("E, ne doit jamais arriver, " + e);
        }
    }
}
```

La classe UnEcrivain

4

```
class UnEcrivain1 implements Runnable {
    private int monId;
    private int nblecteurs;
    private final TableauNoir1 tn;
    private Random generateur = new Random();
    private static int cpt = 1;
    private final static Object mutex = new Object();

    public void run() {
        boolean retenter;
        this.trace("initialisé");
        for (int i = 1; i <= 3 ; i++) {
            int dest = generateur.nextInt(nblecteurs) + 1;
            int msg = generateur.nextInt(1000);
            this.trace("envoie " + msg + " à " + dest + " (N° " + i + ")");
            retenter = true;
            while (retenter) { // boucle d'écriture
                try {
                    tn.ecrireUnMessage(msg, dest);
                    retenter = false;
                }
                catch (Exception e) {
                    this.trace("je n'ai pu écrire (" + e + ")");
                }
                this.attendre();
            }
        }
        this.trace("adieu monde cruel");
    }
}
```

La classe UnLecteur

5

```
class UnLecteur1 implements Runnable {

    private int monId;
    private TableauNoir1 tn;

    private static int cpt = 1;
    private final static Object mutex = new Object();

    UnLecteur1 (TableauNoir1 tab) {
        synchronized (mutex) {
            monId = cpt++;
        }
        tn = tab;
    }

    private void trace (String m) {
        System.out.println ("L " + monId + " : " + m);
        Thread.yield(); // rendre la commutation possible
    }

    private void attendre () {
        try {
            Thread.sleep(1000);
        }
        catch (Exception e2) {
            System.out.println ("L, ne doit jamais arriver, " + e2);
        }
    }
}
```

La classe UnLecteur

5

```
class UnLecteur1 implements Runnable {  
  
    private int monId;  
    private TableauNoir1 tn;  
  
    private static int cpt = 1;  
    private final static Object mutex = new Object();  
  
    public void run() {  
        this.trace("initialisé");  
        while (true) {  
            try {  
                int msg = tn.lireUnMessage(monId);  
                this.trace("j'ai reçu " + msg);  
            }  
            catch (Exception e) {  
                this.trace("rien pour moi (" + e + ")");  
                this.attendre();  
            }  
        }  
    }  
}
```

La classe TableauNoir

6

```
class TableauNoir1 {
    private int destinataire[];
    private int message[];
    private int maTaille;
    private final ReentrantReadWriteLock rwl =
        new ReentrantReadWriteLock(true); // activer l'équité

    TableauNoir1(int taille) {
        maTaille = taille;
        destinataire = new int[maTaille];
        message = new int[maTaille];
        for (int i = 0 ; i < maTaille; i++) {
            destinataire[i] = -1; // pas de destinataire
        }
    }
}
```

La classe TableauNoir

6

```
class TableauNoir1 {
    private int destinataire[];
    private int message[];
    private int maTaille;
    private final ReentrantReadWriteLock rwl =
        new ReentrantReadWriteLock(true); // activer l'équité

    public void ecrireUnMessage (int msg, int dest) throws TropPleinException {
        int i;
        rwl.writeLock().lock();
        try {
            for (i = 0 ; i < maTaille; i++) {
                if (destinataire[i] == -1) {
                    destinataire[i] = dest;
                    message[i] = msg;
                    break;
                }
            }
            if (i == maTaille) {
                throw new TropPleinException("Plus de place pour " + dest);
            }
        }
        finally {
            rwl.writeLock().unlock();
        }
    }
}
```

La classe TableauNoir

6

```
class TableauNoir1 {
    private int destinataire[];
    private int message[];
    private int maTaille;
    private final ReentrantReadWriteLock rwl =
        new ReentrantReadWriteLock(true); // activer l'équité

    public int lireUnMessage (int moi) throws RienPourMoiException {
        int valret = 0; // inutile, mais sinon java râle
        int i;
        rwl.readLock().lock();
        try {
            for (i = 0 ; i < maTaille; i++) {
                if (destinataire[i] == moi) {
                    valret = message[i];
                    break; // sortir de la boucle
                }
            }
        }
        finally {
            rwl.readLock().unlock();
        }
        if (i < maTaille) {
            rwl.writeLock().lock();
            destinataire[i] = -1;
            rwl.writeLock().unlock();
        } else {
            throw new RienPourMoiException("Rien pour " + moi);
        }
        return valret;
    }
}
```

La classe TableauNoir

6

```
class TableauNoir1 {
    private int destinataire[];
    private int message[];
    private int maTaille;
    private final ReentrantReadWriteLock rwl =
        new ReentrantReadWriteLock(true); // activer l'équité

    public int lireUnMessage (int moi) throws Rien
    {
        int valret = 0; // inutile, mais sinon jav
        int i;
        rwl.readLock().lock();
        try {
            for (i = 0 ; i < maTaille; i++) {
                if (destinataire[i] == moi) {
                    valret = message[i];
                    break; // sortir de la boucle
                }
            }
        }
        finally {
            rwl.readLock().unlock();
        }
        if (i < maTaille) {
            rwl.writeLock().lock();
            destinataire[i] = -1;
            rwl.writeLock().unlock();
        } else {
            throw new RienPourMoiException("Rien pour " + moi);
        }
        return valret;
    }
}
```

Usage strict des verrous
Lecture et modification

Types de variables
partagée (sur le tas)
locale (sur la pile)

La «classe principale»

7

```
public class LecteursEtEcrivains1 {

    static int tailleFile;
    static int nbLecteurs;
    static int nbEcrivains;

    public static void main(String []args) {
        try {
            tailleFile = Integer.parseInt(args[0]);
            nbLecteurs = Integer.parseInt(args[1]);
            nbEcrivains = Integer.parseInt(args[2]);
        }
        catch (ArrayIndexOutOfBoundsException e) {
            System.err.println ("Arguments requis, taille de la file, nombre de lecteurs, nombre d'écrivains.");
            return;
        }

        TableauNoir1 leTableau = new TableauNoir1(tailleFile);

        for (int i = 0; i < nbEcrivains ; i++) {
            new Thread(new UnEcrivain1(leTableau, nbLecteurs)).start();
        }
        for (int i = 0; i < nbLecteurs ; i++) {
            new Thread(new UnLecteur1(leTableau)).start();
        }
    }
}
```

La «classe principale»

7

```
public class LecteursEtEcrivains1 {
```

```
$ java LecteursEtEcrivains1 2 1 2
E 1 : initialisé
E 2 : initialisé
E 1 : envoie 76 à 1 (N° 1)
E 2 : envoie 478 à 1 (N° 1)
L 1 : initialisé
L 1 : j'ai reçu 76
L 1 : j'ai reçu 478
L 1 : rien pour moi (RienPourMoi: Rien pour 1)
E 2 : envoie 613 à 1 (N° 2)
E 1 : envoie 362 à 1 (N° 2)
L 1 : j'ai reçu 613
L 1 : j'ai reçu 362
L 1 : rien pour moi (RienPourMoi: Rien pour 1)
E 2 : envoie 191 à 1 (N° 3)
E 1 : envoie 534 à 1 (N° 3)
E 2 : adieu monde cruel
E 1 : adieu monde cruel
L 1 : j'ai reçu 191
L 1 : j'ai reçu 534
L 1 : rien pour moi (RienPourMoi: Rien pour 1)
L 1 : rien pour moi (RienPourMoi: Rien pour 1)
L 1 : rien pour moi (RienPourMoi: Rien pour 1)
```

^C

```
$ java LecteursEtEcrivains1 2 3 2
E 1 : initialisé
E 2 : initialisé
E 1 : envoie 567 à 3 (N° 1)
E 2 : envoie 344 à 1 (N° 1)
L 1 : initialisé
L 2 : initialisé
L 1 : j'ai reçu 344
L 3 : initialisé
L 1 : rien pour moi (RienPourMoi: Rien pour 1)
L 2 : rien pour moi (RienPourMoi: Rien pour 2)
L 3 : j'ai reçu 567
L 3 : rien pour moi (RienPourMoi: Rien pour 3)
E 2 : envoie 196 à 2 (N° 2)
E 1 : envoie 199 à 2 (N° 2)
E 1 : envoie 965 à 3 (N° 3)
L 1 : rien pour moi (RienPourMoi: Rien pour 1)
E 2 : envoie 424 à 1 (N° 3)
L 2 : j'ai reçu 196
L 3 : rien pour moi (RienPourMoi: Rien pour 3)
E 2 : je n'ai pu écrire (TropPlein: Plus de place pour 1)
L 2 : j'ai reçu 199
L 2 : rien pour moi (RienPourMoi: Rien pour 2)
E 1 : adieu monde cruel
L 3 : j'ai reçu 965
L 1 : j'ai reçu 424
L 3 : rien pour moi (RienPourMoi: Rien pour 3)
L 1 : rien pour moi (RienPourMoi: Rien pour 1)
E 2 : adieu monde cruel
L 2 : rien pour moi (RienPourMoi: Rien pour 2)
```

^C

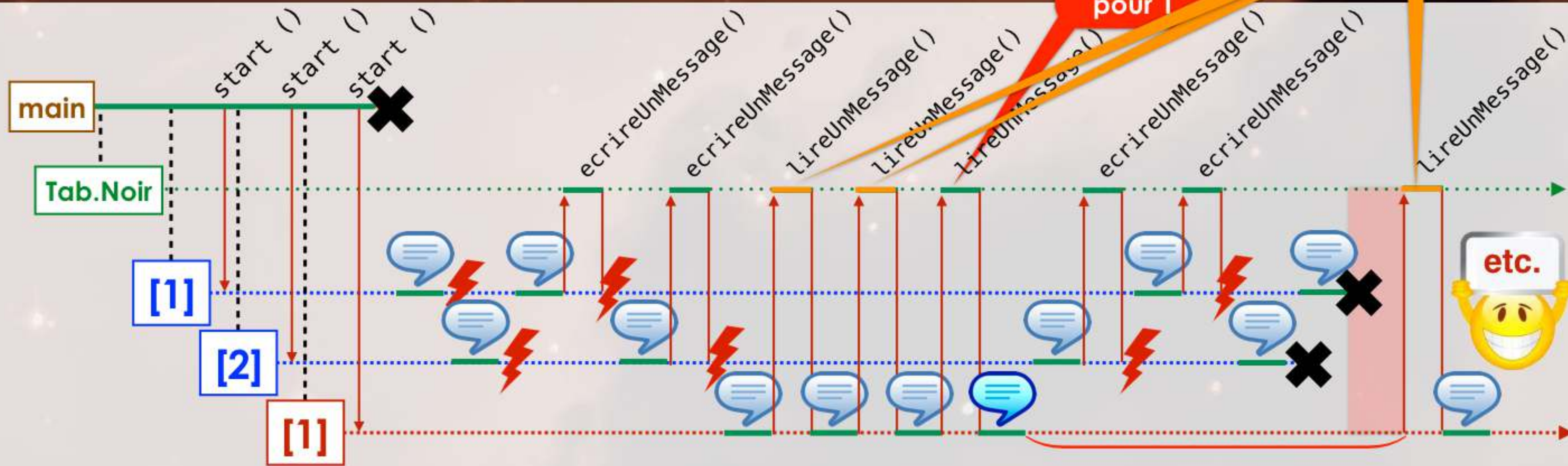
Analyse fine d'une exécution partielle

(2 messages envoyés par écrivain seulement)

verrou R
puis W

plus rien
pour 1

8



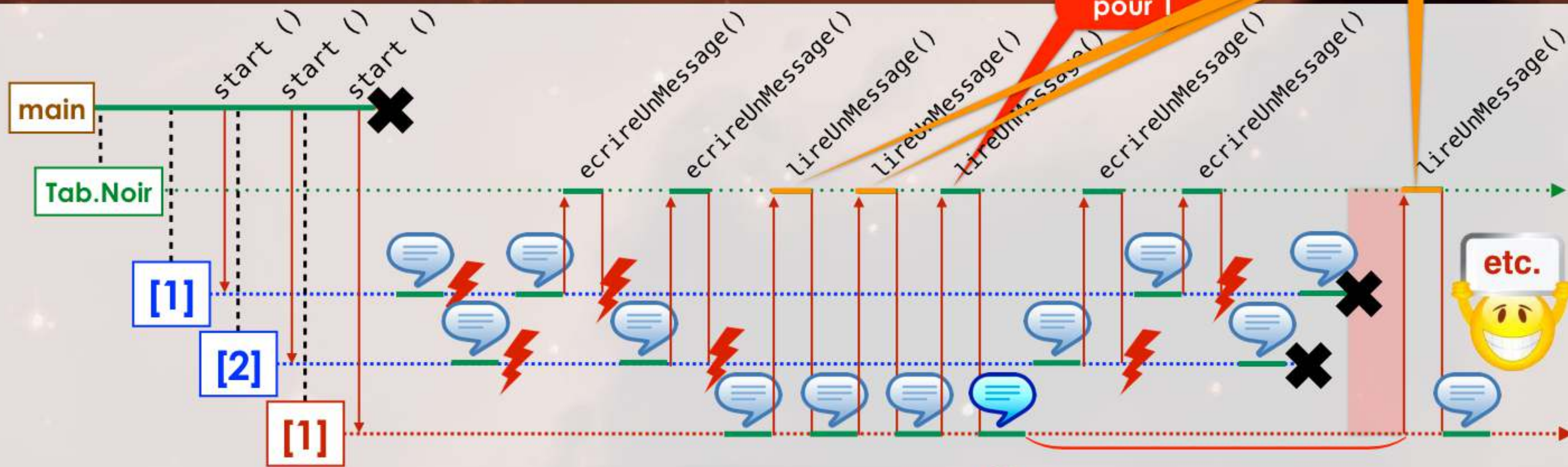
```
$ java LecteursEtEcrivains1 2 1 2
E 1 : initialisé
E 2 : initialisé
E 1 : envoie 76 à 1 (N° 1)
E 2 : envoie 478 à 1 (N° 1)
L 1 : initialisé
L 1 : j'ai reçu 76
L 1 : j'ai reçu 478
L 1 : rien pour moi (RienPourMoi: Rien pour 1)
E 2 : envoie 191 à 1 (N° 2)
E 1 : envoie 534 à 1 (N° 2)
E 2 : adieu monde cruel
E 1 : adieu monde cruel
L 1 : j'ai reçu 191
...
```

Analyse fine d'une exécution partielle

(2 messages envoyés par écrivain seulement)

verrou R
puis W

8



Remarque 2

Pas de blocage sur écriture

Remarque 1

Communication par exception

Suite facile à imaginer

Lecture de la dernière valeur
+ boucle infinie

```
L 1 : j'ai reçu 76  
L 1 : j'ai reçu 478  
L 1 : rien pour moi (RienPourMoi)  
E 2 : envoie 191 à 1 (N° 2)  
E 1 : envoie 534 à 1 (N° 2)  
E 2 : adieu monde cruel  
E 1 : adieu monde cruel  
L 1 : j'ai reçu 191  
...
```



etc.

En guise de conclusion...

9

 Facile non?



 Mais nous allons voir d'autres variantes

- Séquences suivantes
- Illustrer différents aspects
 - ▶ des ReentrantReadWriteLock
 - ▶ de la programmation concurrente